Basics of Applied Numerical Analysis – HW3

Anastasia Golubkova

17, October 2026

The code for the problems (which is also typed here) is available here: `https://github.com/Nastya-golubkova/BoANA/blob/main/hw3/hw3.ipynb`

Problem 1. Code functions for Euler, Trapezoid, and RK4 methods. For the problem from Example 6.25 (Sauer, p. 334) use these function to demonstrate dependence of local and global truncation error for every of these methods on a grid step (in one plot described below)

$$h = 0.5, 0.3, 0.1, 0.03, 0.05, 0.01, 0.003, 0.005, 0.001, 0.0005.$$

Compare with derived formulas and discuss. For this use one (!) plot for all methods for the global error and the another plot for the local error in coordinates log(error) vs log(h). In order to find "exact solution" you can use implicit solution in the additional notes (it could be beneficial to get rid of denominator and perhaps to get rid of fractional power), resolved by any of the root finding methods you can find.

Solution.

```
1
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy.optimize import brentq
5
6
7  x0, T = 0.0, 3.0
8  y0 = 0.5
9
10 steps = [0.5, 0.3, 0.1, 0.03, 0.05,
11          0.01, 0.003, 0.005, 0.001, 0.0005]
12
13
14 def f(x, y):
15     return y + 8*y**2 - 9*y**3
16
17 def exact_y(x):
18     if x == 0:
19         return y0
20
21     def residual(y):
22         return (
23             10*np.log(y)
24             - np.log(1-y)
25             - 9*np.log(1+9*y)
26             - 10*x
27             + 9*np.log(11)
28         )
29
30     return brentq(
31         residual, 0.5, 1 - 1e-14, xtol=1e-15, rtol=4*np.finfo(float).eps)
32
33
34
35 def euler_step(x, y, h):
36     return y + h*f(x, y)
37
```

```
38
39 def trapezoid_step(x, y, h):
40     k1 = f(x, y)
41     k2 = f(x + h, y + h*k1)
42     return y + h/2*(k1 + k2)
43
44
45 def rk4_step(x, y, h):
46     k1 = f(x, y)
47     k2 = f(x + h/2, y + h*k1/2)
48     k3 = f(x + h/2, y + h*k2/2)
49     k4 = f(x + h, y + h*k3)
50
51     return y + h/6*(k1 + 2*k2 + 2*k3 + k4)
52
53
54 methods = {
55     "Euler": euler_step,
56     "Explicit trapezoid": trapezoid_step,
57     "RK4": rk4_step,
58 }
59
60 global_errors = {name: [] for name in methods}
61 local_errors = {name: [] for name in methods}
62
63 for h in steps:
64     N = round((T - x0)/h)
65     x = np.linspace(x0, T, N + 1)
66
67     y_exact = np.array([exact_y(xi) for xi in x])
68
69     for name, step_method in methods.items():
70
71         y_num = np.zeros(N + 1)
72         y_num[0] = y0
73
74         for i in range(N):
75             y_num[i + 1] = step_method(x[i], y_num[i], h)
76
77         global_error = np.max(np.abs(y_num - y_exact))
78         global_errors[name].append(global_error)
79         one_step_values = np.array([ step_method(x[i], y_exact[i], h) for i in
               range(N)])
80         local_error = np.max(np.abs(one_step_values - y_exact[1:]))
81         local_errors[name].append(local_error)
82
83
84
85 steps = np.array(steps)
86 order = np.argsort(steps)
87 h_sorted = steps[order]
88 fit_mask = (h_sorted >= 0.003) & (h_sorted <= 0.05)
89
90
91 def plot_errors(errors, title):
92     plt.figure(figsize=(9, 6))
93     for name, values in errors.items():
94         values = np.array(values)[order]
95
96         plt.loglog(h_sorted, values, "o-", label=name)
97         slope, intercept = np.polyfit(np.log(h_sorted[fit_mask]), np.log(values
               [fit_mask]), 1)
98         plt.loglog(h_sorted[fit_mask], np.exp(intercept) * h_sorted[fit_mask]**
```

```
            slope, "--", label=f"{name} fit: slope = {slope:.2f}")
        print(f"{title}  -  {name}: slope = {slope:.3f}")

    plt.xlabel("Step size h")
    plt.ylabel("Absolute error")
    plt.title(title)
    plt.grid(True, which="both", alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

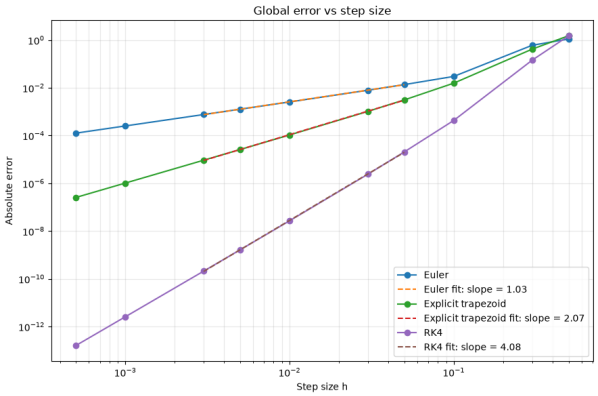

plot_errors(global_errors, "Global error vs step size")

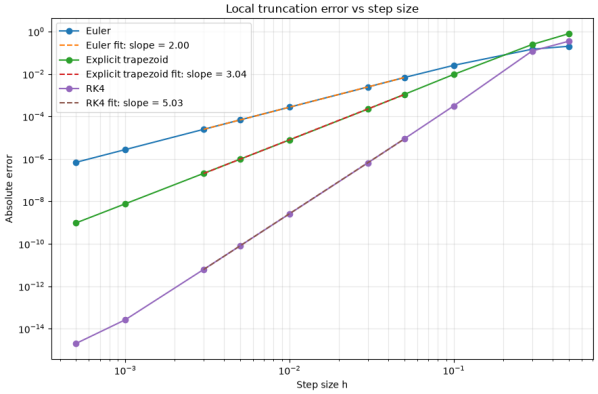
plot_errors(local_errors, "Local truncation error vs step size")


output:

Global error vs step size  -  Euler: slope = 1.026
Global error vs step size  -  Explicit trapezoid: slope = 2.065
Global error vs step size  -  RK4: slope = 4.085


Local truncation error vs step size  -  Euler: slope = 1.997
Local truncation error vs step size  -  Explicit trapezoid: slope = 3.043
Local truncation error vs step size  -  RK4: slope = 5.034
```





The reference solution is computed from the implicit relation

$$10\ln y - \ln(1-y) - 9\ln(1+9y) - 10x + 9\ln 11 = 0$$

by Brent's method with tolerance close to machine precision, so that the error of the "exact" solution is far below the errors being measured.

Theoretical orders. For a method of order p the local truncation error is $\mathcal{O}(h^{p+1})$ and the global error is $\mathcal{O}(h^p)$. Explicitly:

- Euler: $y(x+h) - y_{\text{Euler}} = \frac{h^2}{2} y''(x) + \mathcal{O}(h^3)$, local error $\mathcal{O}(h^2)$, global error $\mathcal{O}(h)$;
- explicit trapezoid: local $\mathcal{O}(h^3)$, global $\mathcal{O}(h^2)$;
- RK4: local $\mathcal{O}(h^5)$, global $\mathcal{O}(h^4)$.

Comparison. The fitted slopes (log–log) are $\approx 1, 2, 4$ for the global error and $\approx 2, 3, 5$ for the local error, which agrees with the formulas above (the fit is done on the asymptotic range of h only, where the error is well above the accuracy of the reference solution).

Large steps. For large time steps the methods may operate outside their asymptotic accuracy and stability regimes. The higher-order methods do not reach their expected asymptotic error reduction at these steps, but RK4 is still more accurate than Euler at $h = 0.3$: the global errors are $1.44 \cdot 10^{-1}$ and $6.06 \cdot 10^{-1}$. The problem is stiff near the equilibrium $y = 1$: $\partial f/\partial y(1) = 1 + 16 - 27 = -10$. The linear stability limits for the real negative eigenvalue $\lambda = -10$ are $h < 0.2$ for Euler and the explicit trapezoid method and $h \lesssim 0.28$ for RK4. The results for these large steps should not be used to estimate the asymptotic orders.

Problem 2. Code two functions for a Backward Euler method using for the first one fixed point iterations and Newton's method for the second one.

Use this function to reproduce Figure 6.22 for Example 6.25 (see above), compare errors (local and global) for Euler, Backward Euler (use Newton's method subroutine of two coded before), Trapezoid, and RK4 methods for steps h = 0.3, h = 0.15, h = 0.03, h = 0.01. Fixed point iterations will fail for h = 0.3, explain why.

Solution.

```python
def dfdy(x, y):
    return 1 + 16*y - 27*y**2


def backward_euler_fixed_point(x, y, h, tol=1e-12, max_iter=100):
    z = y

    for _ in range(max_iter):
        z_new = y + h * f(x + h, z)

        if abs(z_new - z) < tol:
            return z_new

        z = z_new

    raise RuntimeError("Fixed-point iteration did not converge")


def backward_euler_newton(x, y, h, tol=1e-12, max_iter=100):
    z = y + h * f(x, y)

    for _ in range(max_iter):
        F = z - y - h * f(x + h, z)
        dF = 1 - h * dfdy(x + h, z)

        if abs(F) < tol:
            return z

        z_new = z - F/dF

        if abs(z_new - z) < tol:
            if abs(z_new - y - h * f(x + h, z_new)) < 100 * tol:
                return z_new
```

```
        z = z_new


def solve(step_method, h):
    N = round((T - x0) / h)
    x = np.linspace(x0, T, N + 1)
    y = np.empty(N + 1)
    y[0] = y0
    for i in range(N):
        y[i + 1] = step_method(x[i], y[i], h)
    return x, y


x_exact = np.linspace(x0, T, 400)
y_exact_plot = np.array([exact_y(xi) for xi in x_exact])

plt.figure(figsize=(9, 6))
plt.plot(x_exact, y_exact_plot, "k-", lw=2, label="Exact")
for h in [0.3, 0.15, 0.03, 0.01]:
    x, y = solve(backward_euler_newton, h)
    plt.plot(x, y, "o-", ms=3, label=f"Backward Euler, h={h}")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Backward Euler (Newton)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()



h = 0.3
try:
    backward_euler_fixed_point(x0, y0, h)
    print(f"h = {h}: fixed point converged")
except RuntimeError as err:
    z_star = backward_euler_newton(x0, y0, h)
    contraction = abs(h * dfdy(x0 + h, z_star))
    print(err)
    print(f"Newton solution of the first step: z* = {z_star:.6f}")
    print(f"|h * df/dy(z*)| = {contraction:.3f} > 1, so the iteration is not a
        contraction")

steps = [0.3, 0.15, 0.03, 0.01]

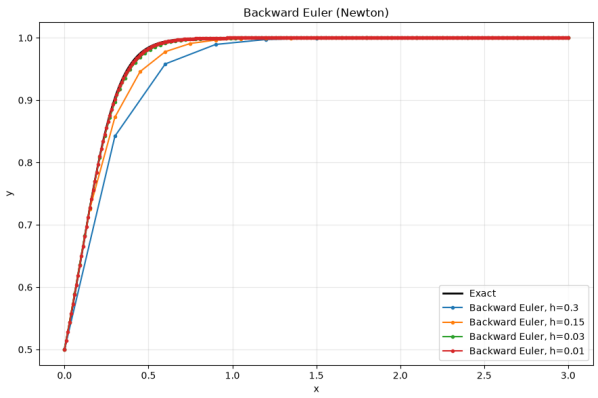
methods = {
    "Euler": euler_step,
    "Backward Euler (Newton)": backward_euler_newton,
    "Explicit trapezoid": trapezoid_step,
    "RK4": rk4_step,
}

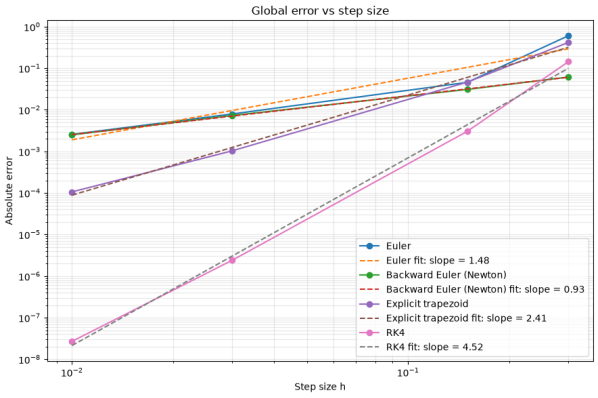
global_errors = {name: [] for name in methods}
local_errors = {name: [] for name in methods}

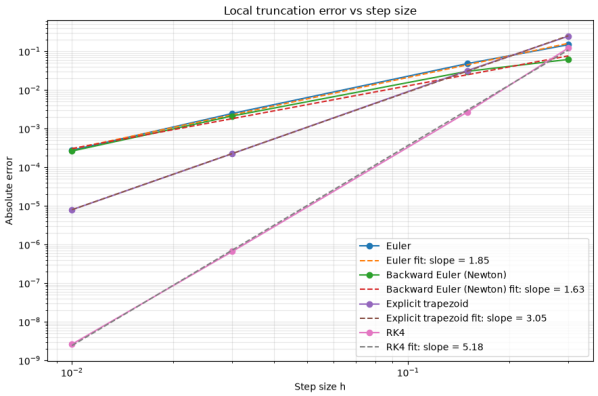
for h in steps:
    x, _ = solve(euler_step, h)
    y_exact = np.array([exact_y(xi) for xi in x])

    for name, step_method in methods.items():
        _, y_num = solve(step_method, h)
```

```
96
97          global_errors[name].append(np.max(np.abs(y_num - y_exact)))
98
99          one_step_values = np.array([
100             step_method(x[i], y_exact[i], h) for i in range(len(x) - 1)
101         ])
102         local_errors[name].append(np.max(np.abs(one_step_values - y_exact[1:]))
                )
103
104
105 steps = np.array(steps)
106 order = np.argsort(steps)
107 h_sorted = steps[order]
108 fit_mask = (h_sorted >= 0.003) & (h_sorted <= 0.05)
109
110 plot_errors(global_errors, "Global error vs step size")
111 plot_errors(local_errors, "Local truncation error vs step size")
112
113
114
115
116 output:
117
118 Fixed-point iteration did not converge
119 Newton solution of the first step: z* = 0.842148
120 |h * df/dy(z*)| = 1.402 > 1, so the iteration is not a contraction
121 Global error vs step size  -  Euler: slope = 1.029
122 Global error vs step size  -  Backward Euler (Newton): slope = 0.978
123 Global error vs step size  -  Explicit trapezoid: slope = 2.072
124 Global error vs step size  -  RK4: slope = 4.093
125
126
127
128 Local truncation error vs step size  -  Euler: slope = 1.991
129 Local truncation error vs step size  -  Backward Euler (Newton): slope = 1.904
130 Local truncation error vs step size  -  Explicit trapezoid: slope = 3.046
131 Local truncation error vs step size  -  RK4: slope = 5.033
```







Why fixed-point iteration fails for $h = 0.3$. One backward Euler step requires solving $z = y_n + hf(x_n + h, z)$. The fixed-point iteration

$$z_{k+1} = y_n + hf(x_n + h, z_k)$$

converges (locally) only if the map is a contraction near the root z^*, i.e. $|h\,\partial f/\partial y(z^*)| < 1$. Here $f(y) = y + 8y^2 - 9y^3$ and $\partial f/\partial y = 1 + 16y - 27y^2$. On the first step ($y_0 = 1/2$, $h = 0.3$) Newton's method gives $z^* \approx 0.842$ and

$$|h\,\partial f/\partial y(z^*)| \approx 1.40 > 1,$$

so the fixed point is repulsive and the iteration does not converge. Moreover, near the equilibrium $y = 1$ we have $|\partial f/\partial y| = 10$, so the fixed-point iteration requires $h < 0.1$ in the later steps as well (for $h = 0.15$ one gets $|hf_y| = 1.5 > 1$ there). Newton's method applied to $F(z) = z - y_n - hf(x_n + h, z) = 0$ does not need the contraction condition: its convergence is local but quadratic, and the Euler predictor $z_0 = y_n + hf(x_n, y_n)$ provides a sufficiently good initial guess.

Comparison of the methods.

h	Euler	Backward Euler	Trapezoid	RK4
0.3	$6.06 \cdot 10^{-1}$	$6.14 \cdot 10^{-2}$	$4.18 \cdot 10^{-1}$	$1.44 \cdot 10^{-1}$
0.15	$4.58 \cdot 10^{-2}$	$3.09 \cdot 10^{-2}$	$4.63 \cdot 10^{-2}$	$3.03 \cdot 10^{-3}$
0.03	$7.95 \cdot 10^{-3}$	$7.31 \cdot 10^{-3}$	$1.03 \cdot 10^{-3}$	$2.41 \cdot 10^{-6}$
0.01	$2.57 \cdot 10^{-3}$	$2.50 \cdot 10^{-3}$	$1.06 \cdot 10^{-4}$	$2.69 \cdot 10^{-8}$

Таблица 1: Global error $\max_n |y_n - y(x_n)|$ on $[0, 3]$.

Backward Euler has the same order as Euler (global order ≈ 1), and for small h its error is comparable with Euler's. For the large step $h = 0.3$, however, it is the most accurate of the four methods, because it is A-stable and is not affected by the stiffness of the problem ($\partial f/\partial y(1) = -10$), whereas the explicit methods are at the edge of or beyond their stability limits. For small h the higher-order methods are far more accurate: the trapezoid method has order 2 and RK4 has order 4.

Problem 3. Compute sequentially forward and inverse Fourier transforms of a Gaussian (choose an interval where difference between maximum and minimum in exact arithmetics would be more than 30

h	Euler	Backward Euler	Trapezoid	RK4
0.3	$1.48 \cdot 10^{-1}$	$6.14 \cdot 10^{-2}$	$2.46 \cdot 10^{-1}$	$1.22 \cdot 10^{-1}$
0.15	$4.79 \cdot 10^{-2}$	$3.03 \cdot 10^{-2}$	$3.16 \cdot 10^{-2}$	$2.66 \cdot 10^{-3}$
0.03	$2.45 \cdot 10^{-3}$	$2.11 \cdot 10^{-3}$	$2.24 \cdot 10^{-4}$	$6.55 \cdot 10^{-7}$
0.01	$2.75 \cdot 10^{-4}$	$2.61 \cdot 10^{-4}$	$7.88 \cdot 10^{-6}$	$2.60 \cdot 10^{-9}$

Таблица 2: Local error (one step from the exact solution, maximum over all steps).

orders of magnitude). Plot initial and resulting (after two transforms) functions in logarithmic on y scale in one figure. Explain the difference. Compute $f'(x)$ from previous part numerically using some FFT library (FFTW is recommended, documentation and the latest version can be found here http://fftw.org, Python interface is available). Plot L2-norm of the error for numerically computed $f'(x)$ as a function of number of sampling points N over the period. Compare with the first and second order finite difference formulae. Discuss the result.

Solution.

```

import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.exp(-x**2)


L = 9.0
N = 4096

x = np.linspace(-L, L, N, endpoint=False)
y = f(x)

y_hat = np.fft.fft(y)

y_rec = np.fft.ifft(y_hat).real

print("f(0) =", f(0))
print("f(9) =", f(9))
print("Exact max/min ratio =", np.exp(L**2))
print("log10(max/min) =", L**2 / np.log(10))


print("Maximum reconstruction error:", np.max(np.abs(y - y_rec)))
relative_error = np.abs(y_rec - y) / y

print("Maximum relative error:", np.max(relative_error))

plt.figure(figsize=(9, 5))

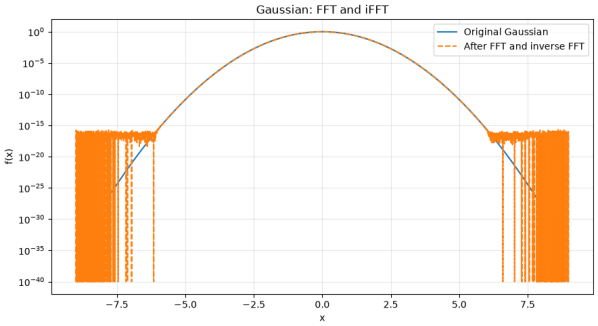
plt.semilogy(x, y, label="Original Gaussian")
plt.semilogy(x, np.maximum(np.abs(y_rec), 1e-40), "--",label="After FFT and
    inverse FFT")

plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Gaussian: FFT and iFFT")
plt.grid(True, which="both", alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()



output:
```

```
f(0) = 1.0
f(9) = 6.639677199580735e-36
Exact max/min ratio = 1.5060973145850306e+35
log10(max/min) = 35.1778530341634
Maximum reconstruction error: 3.3306690738754696e-16
Maximum relative error: 2.1413187855655264e+19
```



The Gaussian e^{-x^2} on $[-9, 9]$ varies from 1 to $e^{-81} \approx 6.6 \cdot 10^{-36}$, i.e. by ≈ 35 orders of magnitude. The forward and inverse FFT are exact in exact arithmetic, but in floating point they produce an *absolute* error of order $\varepsilon \max |f| \approx 10^{-16}$, not a relative one. Hence the maximal absolute error is about $3 \cdot 10^{-16}$, but wherever $f(x) \lesssim 10^{-16}$ (roughly for $|x| \gtrsim 6$) the reconstruction is dominated by roundoff. The relative error reaches $2 \cdot 10^{19}$, and the reconstructed values may even be negative. The plot shows the absolute value of the reconstruction, so those values appear as positive magnitudes rather than disappearing. Higher-precision arithmetic would be required to preserve the full dynamic range of the Gaussian.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.exp(-x**2)

def df_exact(x):
    return -2 * x * np.exp(-x**2)


L = 9.0


def derivative_fft(y, h):
    N = len(y)
    k = 2 * np.pi * np.fft.fftfreq(N, d=h)
    y_hat = np.fft.fft(y)

    dy_hat = 1j * k * y_hat

    if N % 2 == 0:
        dy_hat[N // 2] = 0.0

    return np.fft.ifft(dy_hat).real

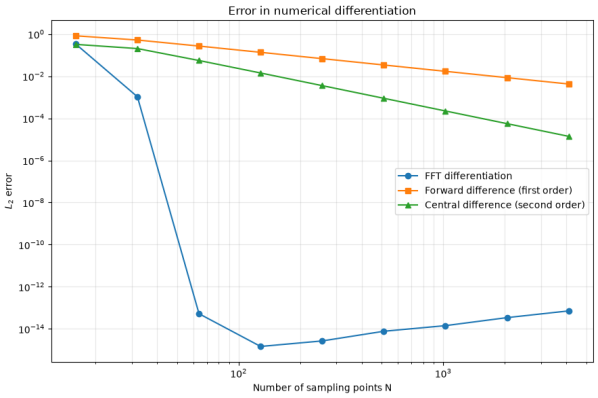

def derivative_forward(y, h):
    return (np.roll(y, -1) - y) / h


def derivative_central(y, h):
    return (np.roll(y, -1) - np.roll(y, 1)) / (2 * h)

```

```
35
36 def L2_error(dy_num, dy_true, h):
37     return np.sqrt(h * np.sum((dy_num - dy_true)**2))
38
39
40 N_values = [16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
41
42 errors_fft = []
43 errors_forward = []
44 errors_central = []
45
46 for N in N_values:
47     h = 2 * L / N
48     x = np.linspace(-L, L, N, endpoint=False)
49
50     y = f(x)
51     dy_true = df_exact(x)
52
53     dy_fft = derivative_fft(y, h)
54     dy_forward = derivative_forward(y, h)
55     dy_central = derivative_central(y, h)
56
57     errors_fft.append(L2_error(dy_fft, dy_true, h))
58     errors_forward.append(L2_error(dy_forward, dy_true, h))
59     errors_central.append(L2_error(dy_central, dy_true, h))
60
61
62 print(f"{'N':>6} {'FFT':>14} {'Forward':>14} {'Central':>14}")
63 for N, e_fft, e_forward, e_central in zip(
64         N_values, errors_fft, errors_forward, errors_central):
65     print(f"{N:6d} {e_fft:14.6e} "
66           f"{e_forward:14.6e} {e_central:14.6e}")
67
68 plt.figure(figsize=(9, 6))
69
70 plt.loglog(N_values, errors_fft, "o-",
71            label="FFT differentiation")
72 plt.loglog(N_values, errors_forward, "s-",
73            label="Forward difference (first order)")
74 plt.loglog(N_values, errors_central, "^-",
75            label="Central difference (second order)")
76
77 plt.xlabel("Number of sampling points N")
78 plt.ylabel(r"$L_2$ error")
79 plt.title("Error in numerical differentiation")
80 plt.grid(True, which="both", alpha=0.3)
81 plt.legend()
82 plt.tight_layout()
83 plt.show()
84
85
86 output:
87
88
89 N              FFT         Forward         Central
90     16   3.344977e-01   8.260145e-01   3.239611e-01
91     32   1.054868e-03   5.221690e-01   2.052778e-01
92     64   5.246082e-14   2.697090e-01   5.561221e-02
93    128   1.466496e-15   1.359663e-01   1.419217e-02
94    256   2.680860e-15   6.812319e-02   3.566468e-03
95    512   7.715583e-15   3.407914e-02   8.927741e-04
96   1024   1.426524e-14   1.704176e-02   2.232659e-04
97   2048   3.420681e-14   8.521156e-03   5.582101e-05
```

```
98   4096    7.144977e-14    4.260612e-03    1.395554e-05
```



We compute $f'(x)$ on a periodic grid with $h = 2L/N$. The finite-difference errors decay as $\mathcal{O}(h)$ (forward) and $\mathcal{O}(h^2)$ (central), which is confirmed by the table: doubling N reduces the errors by factors of ≈ 2 and ≈ 4, respectively.

The spectral (FFT) error decays much faster than any power of h: the function is smooth and effectively periodic, so the error is determined by the decay of its Fourier spectrum, $\hat{f}(k) \sim e^{-k^2/4}$. The error falls from 10^{-3} at $N = 32$ to $5 \cdot 10^{-14}$ at $N = 64$ (at $N = 64$ the largest wavenumber is $k_{\max} = \pi/h \approx 11$, where $e^{-k_{\max}^2/4} \approx 10^{-14}$), and reaches about $1.5 \cdot 10^{-15}$ at $N = 128$. On the last points of the table the FFT error then increases. That rise is consistent with roundoff $\sim \varepsilon$ multiplied by $k_{\max} = \pi/h \propto N$ from differentiation, but it is only the trend on those points, not a law for the whole range of N.

The FFT method is already at roundoff for $N \approx 64$–128. The finite-difference methods are not close to that accuracy on the computed grids, and the two formulae need very different N to get there. The central difference at $N = 4096$ has error about $1.4 \cdot 10^{-5}$. Scaling it as N^{-2} down to 10^{-14} would require

$$N \approx 4096 \sqrt{\frac{1.4 \cdot 10^{-5}}{10^{-14}}} \approx 1.5 \cdot 10^8.$$

The forward difference scales only as N^{-1}, so the same estimate gives on the order of 10^{15} points. We used the FFT from NumPy (pocketfft); FFTW (e.g. via `pyfftw`) would produce the same results up to roundoff.

Problem 4. Show analytically that symmetric second order split-step method is really second order (show that local error behaves as cube of the timestep). Error formula and description of the method are given in https://arxiv.org/pdf/2008.03938, see p.5

Solution.

Consider the evolution equation

$$\frac{du}{dt} = (A + B)u,$$

where A and B are linear operators. The exact solution operator over a single time step h is given by the exponential $e^{h(A+B)}$. The symmetric second-order split-step method (Strang splitting) is defined by the numerical operator:

$$S(h) = e^{\frac{h}{2}A} e^{hB} e^{\frac{h}{2}A}.$$

To show that the method is second-order accurate, we must demonstrate that the local truncation error is $\mathcal{O}(h^3)$. We do this by expanding the exact and numerical operators in Taylor series up to $\mathcal{O}(h^2)$ and showing they match exactly.

First, we expand the individual exponential operators:

$$e^{\frac{h}{2}A} = I + \frac{h}{2}A + \frac{h^2}{8}A^2 + \mathcal{O}(h^3),$$
$$e^{hB} = I + hB + \frac{h^2}{2}B^2 + \mathcal{O}(h^3).$$

Next, we multiply the first two terms, $e^{\frac{h}{2}A}e^{hB}$, keeping only terms up to h^2:

$$\begin{aligned} e^{\frac{h}{2}A}e^{hB} &= \left(I + \frac{h}{2}A + \frac{h^2}{8}A^2\right)\left(I + hB + \frac{h^2}{2}B^2\right) + \mathcal{O}(h^3) \\ &= I + hB + \frac{h^2}{2}B^2 + \frac{h}{2}A + \frac{h^2}{2}AB + \frac{h^2}{8}A^2 + \mathcal{O}(h^3) \\ &= I + \frac{h}{2}(A + 2B) + h^2\left(\frac{1}{8}A^2 + \frac{1}{2}AB + \frac{1}{2}B^2\right) + \mathcal{O}(h^3). \end{aligned}$$

Now, we multiply this result by the second $e^{\frac{h}{2}A}$ from the right:

$$S(h) = \left[I + \frac{h}{2}(A + 2B) + h^2\left(\frac{1}{8}A^2 + \frac{1}{2}AB + \frac{1}{2}B^2\right)\right]\left[I + \frac{h}{2}A + \frac{h^2}{8}A^2\right] + \mathcal{O}(h^3).$$

Expanding the product and collecting terms by powers of h:

- h^0: $I \cdot I = I$.
- h^1: $I \cdot \frac{h}{2}A + \frac{h}{2}(A + 2B) \cdot I = \frac{h}{2}A + \frac{h}{2}A + hB = h(A + B)$.
- h^2:

$$\begin{aligned} & I \cdot \frac{h^2}{8}A^2 + \frac{h}{2}(A + 2B) \cdot \frac{h}{2}A + h^2\left(\frac{1}{8}A^2 + \frac{1}{2}AB + \frac{1}{2}B^2\right) \cdot I \\ &= \frac{h^2}{8}A^2 + \frac{h^2}{4}A^2 + \frac{h^2}{2}BA + \frac{h^2}{8}A^2 + \frac{h^2}{2}AB + \frac{h^2}{2}B^2 \\ &= h^2\left(\frac{4}{8}A^2\right) + \frac{h^2}{2}(AB + BA) + \frac{h^2}{2}B^2 \\ &= \frac{h^2}{2}A^2 + \frac{h^2}{2}(AB + BA) + \frac{h^2}{2}B^2 \\ &= \frac{h^2}{2}(A^2 + AB + BA + B^2) = \frac{h^2}{2}(A + B)^2. \end{aligned}$$

Combining these, the numerical operator expands to:

$$S(h) = I + h(A + B) + \frac{h^2}{2}(A + B)^2 + \mathcal{O}(h^3).$$

On the other hand, the Taylor expansion of the exact evolution operator is:

$$e^{h(A+B)} = I + h(A + B) + \frac{h^2}{2}(A + B)^2 + \mathcal{O}(h^3).$$

Comparing the two expansions, we see that they are identical up to the second order. Therefore, the difference between the exact and numerical operators is:

$$e^{h(A+B)} - S(h) = \mathcal{O}(h^3).$$

This proves that the local truncation error of the symmetric split-step method is $\mathcal{O}(h^3)$. Since the total integration time T requires $N = T/h$ steps, the global error accumulates as:

$$E_{\text{global}} = N \times \mathcal{O}(h^3) = \frac{T}{h}\mathcal{O}(h^3) = \mathcal{O}(h^2).$$

The symmetric split-step method is indeed second-order accurate.

To show that the order is *exactly* two, we must check that the $\mathcal{O}(h^3)$ term does not vanish. The Baker–Campbell–Hausdorff formula for the symmetric product gives

$$e^{\frac{h}{2}A}e^{hB}e^{\frac{h}{2}A} = \exp\Big(h(A + B) + h^3\Big(\tfrac{1}{12}[B,[B,A]] - \tfrac{1}{24}[A,[A,B]]\Big) + \mathcal{O}(h^5)\Big),$$

and therefore

$$e^{h(A+B)} - S(h) = h^3\Big(\tfrac{1}{24}[A,[A,B]] - \tfrac{1}{12}[B,[B,A]]\Big) + \mathcal{O}(h^4).$$

For $[A, B] \neq 0$ (e.g. the dispersion operator and the nonlinear term of the NLSE) this term is generally non-zero. Hence the local error behaves exactly as h^3. (The coefficients agree with the error formula on p. 5 of the paper.)

Passing from the local to the global error requires stability of the scheme: the sum of the local errors over $N = T/h$ steps does not grow if the step operators are bounded in norm, which holds for the NLSE, where both the linear and the nonlinear substeps are unitary (norm-preserving). Hence $E_{\text{global}} = \mathcal{O}(h^2)$. The series above are understood formally (for sufficiently smooth solutions), since A is an unbounded differential operator.

Problem 5. Implement the first and second order split-step methods for the NLSE using formulae from `https://arxiv.org/pdf/2008.03938`, page 5 (the first order split-step method is described in `https://arxiv.org/abs/1611.10008`, see pp. 4-5). For the initial condition described in Section 6.2 of `https://arxiv.org/pdf/2008.03938` (from now on all the references are given to this paper), page 9, perform simulation until the soliton will move through the period. It is supposed to move without change of shape according to formula Eq. (5), so this is your exact solution. Plot an error in L_{inf} norm at some given moment of time for both methods as a function of a time step. Demonstrate orders of both methods (essentially, you need to plot something similar to the left panel of the Figure 1, only there will be two curves with different slopes, 4-5 points would be enough).

Solution.

```
1
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6
7  L = 25 * np.pi
8  N = 1024
9  x = np.linspace(-L, L, N, endpoint=False)
10 dx = 2 * L / N
11
12
13 k = 2 * np.pi * np.fft.fftfreq(N, d=dx)
14
15
16 lam = 2.0
17 v = 2.0
18 x0 = 0.0
19 phi0 = 0.0
20
21
22 T = 2 * L / v
23
24
25 def exact_solution(x, t):
26     phase = 0.5 * v * x + (lam - v**2 / 4) * t + phi0
27     d = (x - v * t - x0 + L) % (2 * L) - L      # this line must be present
28     return np.sqrt(2 * lam) * np.exp(1j * phase) / np.cosh(np.sqrt(lam) * d)
29
30                                     # not 0.37 * T
31
32
33 def linear_step(phi, dt):
34
35     phi_hat = np.fft.fft(phi)
36     phi_hat *= np.exp(-1j * k**2 * dt)
37     return np.fft.ifft(phi_hat)
38
39
40 def nonlinear_step(phi, dt):
41
42     return np.exp(1j * np.abs(phi)**2 * dt) * phi
43
```

```
44
45  def split_step_first_order(phi, dt):
46
47      phi = nonlinear_step(phi, dt)
48      phi = linear_step(phi, dt)
49      return phi
50
51
52  def split_step_second_order(phi, dt):
53
54      phi = linear_step(phi, dt / 2)
55      phi = nonlinear_step(phi, dt)
56      phi = linear_step(phi, dt / 2)
57      return phi
58
59
60  T = 2 * L / v
61  T_err = T
62
63  dt_values = []
64  errors_first = []
65  errors_second = []
66
67  dt_targets = [
68      0.003,
69      0.0015,
70      0.00075,
71      0.000375,
72      0.0001875,
73      0.00009375,
74  ]
75
76  dt_first_only = [
77      0.000046875,
78      0.0000234375,
79      0.00001171875,
80  ]
81
82  for dt_target in dt_targets:
83      n_steps = round(T_err / dt_target)
84      dt = T_err / n_steps
85
86      phi_first = exact_solution(x, 0.0)
87      phi_second = phi_first.copy()
88
89      for _ in range(n_steps):
90          phi_first  = split_step_first_order(phi_first, dt)
91          phi_second = split_step_second_order(phi_second, dt)
92
93      phi_exact = exact_solution(x, T_err)
94
95      dt_values.append(dt)
96      errors_first.append(np.max(np.abs(phi_first - phi_exact)))
97      errors_second.append(np.max(np.abs(phi_second - phi_exact)))
98
99  for dt_target in dt_first_only:
100     n_steps = round(T_err / dt_target)
101     dt = T_err / n_steps
102
103     phi_first = exact_solution(x, 0.0)
104     for _ in range(n_steps):
105         phi_first = split_step_first_order(phi_first, dt)
106
```

```
    phi_exact = exact_solution(x, T_err)
    dt_values.append(dt)
    errors_first.append(np.max(np.abs(phi_first - phi_exact)))

dt_values = np.array(dt_values)
errors_first = np.array(errors_first)
errors_second = np.array(errors_second)
dt_second = dt_values[:len(errors_second)]

p_first = np.log(errors_first[:-1] / errors_first[1:]) / np.log(
    dt_values[:-1] / dt_values[1:]
)
p_second = np.log(errors_second[:-1] / errors_second[1:]) / np.log(
    dt_second[:-1] / dt_second[1:]
)

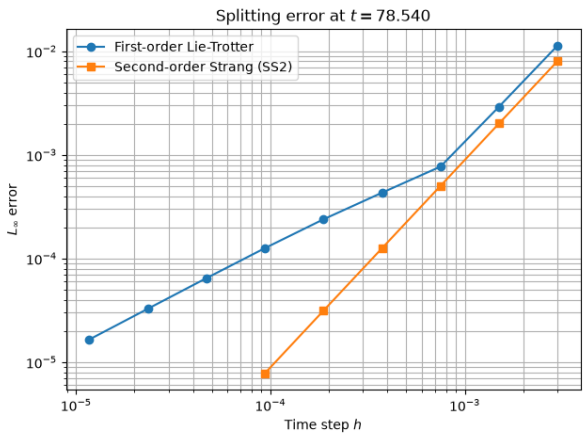
print("First-order errors:", errors_first)
print("SS2 errors:", errors_second)
print("Global orders, first-order method:", p_first)
print("Global orders, SS2:", p_second)


plt.loglog(dt_values, errors_first, "o-", label="First-order Lie-Trotter")
plt.loglog(dt_second, errors_second, "s-", label="Second-order Strang (SS2)")

plt.xlabel("Time step $h$")
plt.ylabel(r"$L_\infty$ error")
plt.title(f"Splitting error at $t={T_err:.3f}$")
plt.grid(True, which="both")
plt.legend()
plt.show()


output:

First-order errors: [0.01135442 0.00291934 0.00077155 0.0004319  0.0002392
    0.00012614
 0.00006472 0.00003281 0.00001658]
SS2 errors: [8.04082058e-03 2.01036057e-03 5.02596687e-04 1.25642759e-04
 3.13991688e-05 7.82648172e-06]
Global orders, first-order method: [1.95954222 1.91981142 0.83704322 0.85251638
    0.92318462
 0.9627     0.9804     0.9844]
Global orders, SS2: [1.99988846 1.99998122 2.00007363 2.00053614 2.00429055]
```



We implemented the first-order Lie–Trotter splitting $S_1(h) = e^{hL}e^{hN}$ and the second-order Strang

splitting $S_2(h) = e^{\frac{h}{2}L} e^{hN} e^{\frac{h}{2}L}$ for the NLSE $i\varphi_t + \varphi_{xx} + |\varphi|^2\varphi = 0$, where L is the linear (dispersive) part, solved exactly in Fourier space, and N is the nonlinear part, solved exactly in physical space. The initial condition is the soliton of Eq. (5), which moves without change of shape, so Eq. (5) is also the exact solution. The simulation is run until the soliton has passed through the whole period, $t = T = 2L/v$, and the L_∞ error is measured at this time.

Since the error is accumulated over $N = T/\Delta t$ steps up to a fixed time T, the observed rates are the *global* orders of accuracy (the local error of a single step would be one order higher). The orders computed as $p \approx \log(E_1/E_2)/\log(\Delta t_1/\Delta t_2)$ are

- first-order method: 1.96, 1.92, 0.84, 0.85, 0.92, 0.96, 0.98, 0.98,
- Strang method: $p \approx 2$ (values: [1.99988846 1.99998122 2.00007363 2.00053614 2.00429055]).

For the Strang method the measured rates stay very close to 2 over the whole range. For the first-order method the rate first drops from about 2 toward 1, and on the three finer steps, $\Delta t \approx 4.7\cdot10^{-5}$, $2.3\cdot10^{-5}$ and $1.2\cdot10^{-5}$, it settles near 1 (0.96, 0.98, 0.98). The errors there are $6.47\cdot10^{-5}$, $3.28\cdot10^{-5}$ and $1.66\cdot10^{-5}$.

Bonus Problem Implement the Hamiltonian integration scheme (see p. 6, Eq. (17) of arXiv:2008.03938). As a convergence conditions instead of Eq. (18) use relative error in L_∞ norm. Choose time-step according to Eq. (19), but get a relative error of the order of 10-11 in 4-6 iterations. Simulate the propagation of a soliton as before. Demonstrate that the method is second order in time. Compare power (number of particles) and energy (see Eq.(2)) conservation in both methods on one picture. Essentially, you need to reproduce both panels of Figure 1 (p. 9) of the paper.

Solution

We consider the focusing nonlinear Schrödinger equation

$$i\Phi_t + \Phi_{xx} + |\Phi|^2\Phi = 0,$$

with periodic boundary conditions on the interval $[-L, L]$.

The spatial domain is discretized using a uniform grid with $N = 1024$ points and spacing $\Delta x = 2L/N$. Spatial derivatives are evaluated using the Fourier spectral method.

The initial condition is a one-soliton solution,

$$\Phi(x,t) = \frac{\sqrt{2\lambda}\, e^{i\left(\frac{v}{2}x + (\lambda - \frac{v^2}{4})t + \phi_0\right)}}{\cosh\left(\sqrt{\lambda}(x - vt - x_0)\right)}.$$

We use $L = 25\pi$, $\lambda = 2$, $v = 2$, $x_0 = \phi_0 = 0$, and $T = 2L/v = 25\pi$. Periodic wrapping is applied when evaluating the soliton profile on the finite spatial interval.

The Hamiltonian Integration Method (HIM) is based on an implicit time discretization. The resulting nonlinear equation for the solution at the next time step is solved using fixed-point iterations.

At each time step the fixed-point iteration (17) is started from one forward Euler step,

$$\Phi^{n+1,0} = \Phi^n + i\Delta t\,(\Phi^n_{xx} + |\Phi^n|^2\Phi^n),$$

as stated after Eq. (18) of the paper. Starting from Φ^n itself reaches the same tolerance, but takes several more iterations. The update itself is Eq. (17): the linear part is applied in Fourier space and the nonlinear term $(\Phi^{n+1,s} + \Phi^n)(|\Phi^{n+1,s}|^2 + |\Phi^n|^2)$ is formed in physical space.

```
def him_step(phi_n, dt, tol=1e-11, max_iter=30):
    dt_max = 2.0 / (np.sqrt(3.0) * np.max(np.abs(phi_n)**2))
    if dt >= dt_max:
        raise ValueError("time step violates Eq. (19)")

    phi_hat_n = np.fft.fft(phi_n)
    denominator = 1.0 + 0.5j * k**2 * dt
    linear_factor = (1.0 - 0.5j * k**2 * dt) / denominator
    nonlinear_factor = 1j * dt / (4.0 * denominator)

    phi_xx = np.fft.ifft(-(k**2) * phi_hat_n)
    phi_iter = phi_n + 1j * dt * (phi_xx + np.abs(phi_n)**2 * phi_n)

    for iteration in range(1, max_iter + 1):
```

```
        nonlinear_term = (
            (phi_iter + phi_n)
            * (np.abs(phi_iter)**2 + np.abs(phi_n)**2)
        )
        phi_hat_next = (
            linear_factor * phi_hat_n
            + nonlinear_factor * np.fft.fft(nonlinear_term)
        )
        phi_next = np.fft.ifft(phi_hat_next)

        rel_error = np.max(np.abs(phi_next - phi_iter)) / np.max(np.abs(
            phi_next))
        if rel_error <= tol:
            return phi_next, iteration, rel_error
        phi_iter = phi_next

    raise RuntimeError("HIM did not converge")


def power(phi):
    return dx * np.sum(np.abs(phi)**2)


def energy(phi):
    phi_x = np.fft.ifft(1j * k * np.fft.fft(phi))
    return dx * np.sum(np.abs(phi_x)**2 - 0.5 * np.abs(phi)**4).real


def solve_him(dt_target, store_invariants=False):
    n_steps = int(np.ceil(T / dt_target))
    dt = T / n_steps
    phi = exact_solution(x, 0.0)
    counts = []
    if store_invariants:
        times = [0.0]
        powers = [power(phi)]
        energies = [energy(phi)]
    for n in range(n_steps):
        phi, n_iter, _ = him_step(phi, dt)
        counts.append(n_iter)
        if store_invariants:
            times.append((n + 1) * dt)
            powers.append(power(phi))
            energies.append(energy(phi))
    if store_invariants:
        return phi, dt, np.array(times), np.array(powers), np.array(energies),
            counts
    return phi, dt, counts
```

The targets 0.008, 0.004, 0.002 and 0.001 do not divide $T = 25\pi$. The outer loop does not keep those values. It takes

$$n = \lceil T/\Delta t_{\text{target}} \rceil, \qquad \Delta t = T/n,$$

and makes exactly n steps of this length, so every run ends at the same time T. The steps actually used are $7.999574 \cdot 10^{-3}$, $3.999991 \cdot 10^{-3}$, $1.999995 \cdot 10^{-3}$ and $9.999977 \cdot 10^{-4}$. The error is compared with the exact solution at this same T.

The iterations are stopped when the relative error in the L_∞ norm satisfies

$$\frac{\|\Phi^{n+1,s+1} - \Phi^{n+1,s}\|_\infty}{\|\Phi^{n+1,s+1}\|_\infty} \le 10^{-11}.$$

The time step is also required to satisfy the convergence condition

$$\Delta t < \frac{2}{\sqrt{3}\max_j |\Phi_j^n|^2},$$

as specified by Eq. (19) of the paper.

To verify the temporal order of accuracy, the simulation is repeated with several time steps while keeping the spatial grid fixed. At the final time T, the relative error is calculated as

$$E(\Delta t) = \frac{\|\Phi_{\text{num}}(T) - \Phi_{\text{exact}}(T)\|_\infty}{\|\Phi_{\text{exact}}(T)\|_\infty}.$$

The observed convergence order between two consecutive time steps is

$$p_{\text{obs}} = \frac{\log\left(E(\Delta t_1)/E(\Delta t_2)\right)}{\log(\Delta t_1/\Delta t_2)}.$$

For a second-order method, the error is expected to satisfy

$$E(\Delta t) = O(\Delta t^2).$$

Consequently, halving the time step should reduce the error by approximately a factor of four in the asymptotic convergence regime.

The computed errors and observed orders are reported in the following table. The two figures below are produced by

```
dt_targets = [0.008, 0.004, 0.002, 0.001]
errors = []
actual_dts = []

for dt_target in dt_targets:
    phi_num, dt, iteration_counts = solve_him(dt_target)
    phi_exact = exact_solution(x, T)
    error = np.max(np.abs(phi_num - phi_exact)) / np.max(np.abs(phi_exact))
    actual_dts.append(dt)
    errors.append(error)

actual_dts = np.array(actual_dts)
errors = np.array(errors)

plt.figure()
plt.loglog(actual_dts, errors, "o-", label="HIM")
reference = errors[0] * (actual_dts / actual_dts[0])**2
plt.loglog(actual_dts, reference, "--", label="O(dt^2)")
plt.xlabel("Time step")
plt.ylabel("Relative error at final time")
plt.grid(True, which="both")
plt.legend()
plt.show()

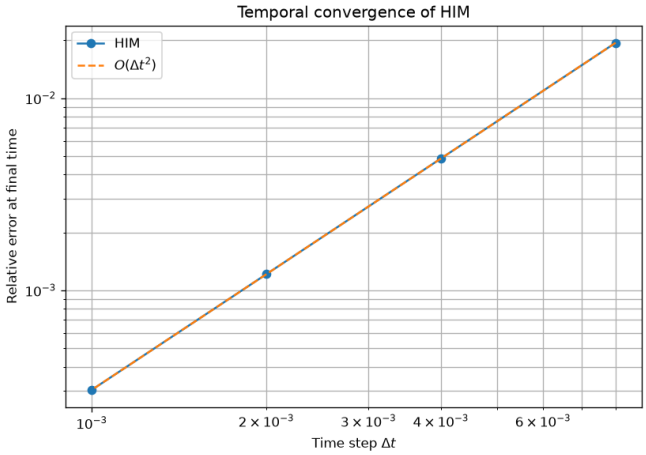
dt_conservation = 0.002
phi_him, dt_him, times, powers_him, energies_him, counts = (
    solve_him(dt_conservation, store_invariants=True)
)
phi = exact_solution(x, 0.0)
powers_ss = [power(phi)]
energies_ss = [energy(phi)]
linear_half = np.exp(-1j * k**2 * dt_him / 2)
for _ in range(len(times) - 1):
    phi = np.fft.ifft(linear_half * np.fft.fft(phi))
    phi = phi * np.exp(1j * np.abs(phi)**2 * dt_him)
    phi = np.fft.ifft(linear_half * np.fft.fft(phi))
    powers_ss.append(power(phi))
    energies_ss.append(energy(phi))

powers_ss = np.array(powers_ss)
energies_ss = np.array(energies_ss)

plt.figure()
```

```
plt.semilogy(times, np.abs(powers_him - powers_him[0]), label="HIM: power")
plt.semilogy(times, np.abs(energies_him - energies_him[0]), label="HIM: energy")
plt.semilogy(times, np.abs(powers_ss - powers_ss[0]), "--", label="Strang: power")
plt.semilogy(times, np.abs(energies_ss - energies_ss[0]), "--", label="Strang: energy")
plt.xlabel("Time")
plt.ylabel("Absolute deviation from initial value")
plt.grid(True, which="both")
plt.legend()
plt.show()
```

Δt	$E(\Delta t)$	p_{obs}	iterations
$8.00 \cdot 10^{-3}$	$1.94 \cdot 10^{-2}$	–	6
$4.00 \cdot 10^{-3}$	$4.85 \cdot 10^{-3}$	2.00	5
$2.00 \cdot 10^{-3}$	$1.21 \cdot 10^{-3}$	2.00	4–5
$1.00 \cdot 10^{-3}$	$3.03 \cdot 10^{-4}$	2.00	4



These steps all satisfy Eq. (19) (here $\max |\Phi|^2 = 4$, so $\Delta t < 2/(4\sqrt{3}) \approx 0.289$). With the forward Euler start, the relative L_∞ residual reaches 10^{-11} in 4 to 6 iterations on every step. Halving Δt reduces the error by a factor of four, so the method is second order in time.

The power (number of particles) and Hamiltonian are defined by

$$N(t) = \int_{-L}^{L} |\Phi(x,t)|^2 \, dx,$$

and

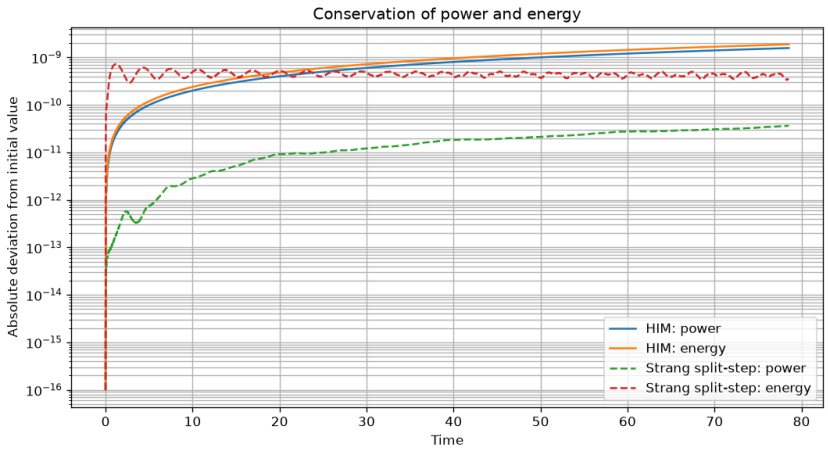
$$H(t) = \int_{-L}^{L} \left(|\Phi_x(x,t)|^2 - \frac{1}{2}|\Phi(x,t)|^4 \right) dx.$$

Both quantities are conserved by the exact solution. In the numerical experiment, the integrals are approximated using the spatial grid, and the derivative Φ_x is computed spectrally.

For each method, the absolute deviations from the initial values are monitored:

$$\Delta N(t) = |N(t) - N(0)|, \qquad \Delta H(t) = |H(t) - H(0)|.$$

The HIM results are compared with the second-order Strang method on the same grid and up to the same final time, at $\Delta t = 2 \cdot 10^{-3}$ (mean number of HIM iterations 4.75, maximum 5).



At $t = T$ the deviations from the initial values are

	$\lvert\Delta N\rvert$	$\lvert\Delta H\rvert$
HIM	$1.58 \cdot 10^{-9}$	$1.89 \cdot 10^{-9}$
Strang	$3.64 \cdot 10^{-11}$	$3.56 \cdot 10^{-10}$.

Both methods keep the power close to its initial value over the whole interval. HIM also keeps the energy at the level of the iteration tolerance. At this time step the Strang method keeps the energy as well; the drift seen for larger Δt is much smaller here.